Covariance Recovery on Bayes Trees via Steiner Subtrees and Solve-Based Inversion

Frank Dellaert - with heavy AI assistance

School of Interactive Computing, Georgia Institute of Technology
dellaert.github.io

Abstract

Bayes-tree covariance recovery has existed for many years in the open-source GTSAM library (Dellaert, 2012; Dellaert and Kaess, 2017), but the recursive mechanism itself has not previously been written up. We present the first account of the one-variable and two-variable mechanisms already present there. For a single queried variable, the Bayes tree $\mathcal{T}$ localizes the marginal to one rootward path through recursive separator marginals. For a pair query, the same recursion localizes the joint marginal to the two rootward paths meeting at the lowest common ancestor, together with exact branch-summary conditionals along those paths. We refer to those already-implemented branch summaries as shortcuts. We then introduce the new structural contribution: for an arbitrary query set $\mathcal{Q}$, the exact support is the Steiner subtree $\mathcal{T}_\mathcal{Q}$ spanning the queried cliques, so all off-subtree descendants can be pruned and the existing shortcut idea can be generalized to compress internal non-branching chains. As a secondary numerical contribution, we replace dense inversion on the reduced problem by solve-based covariance extraction and block selected inversion on the reduced square-root factor $R_\mathcal{Q}$. Throughout, "exact" means exact for the fixed linearized Gaussian posterior represented by the Bayes tree. The result is a query-local account of covariance recovery that separates the long-standing Bayes-tree recursion from the new arbitrary-query Steiner generalization built on top of it.

1 Introduction

We introduced Bayes trees as a representation for sparse Gaussian posteriors $p(x)$ in robotics (Dellaert and Kaess, 2017). Bayes trees are essentially directed clique trees: they orient the clique-tree constructions familiar from probabilistic graphical models (Koller and Friedman, 2009) and sparse linear algebra (Blair and Peyton, 1993), and attach a conditional density to each clique. In earlier Bayes-tree work (Kaess et al., 2010, 2012), we developed this directed clique-tree view as an interpretation of multifrontal elimination. It encodes the Gaussian posterior $p(x)$ as a tree of conditional densities and provides a probabilistic interpretation of sparse square-root factorization. Each clique stores a conditional density of frontal variables given separator variables, so the full posterior $p(x)$ is represented exactly while preserving the elimination structure of the factorization.

We seek to recover marginal covariance blocks without ever forming the global covariance matrix. Let $x \in \mathbb{R}^n$ be a Gaussian random vector with density

$$p(x) \propto \exp\left(-\frac{1}{2}\|Rx - d\|^2\right), \tag{1}$$

where R is block upper triangular after elimination. The information and covariance matrices Λ and Σ are

$$\Lambda = R^\top R, \qquad \Sigma = \Lambda^{-1}. \tag{2}$$

Let $\mathcal{X}$ denote the collection of multivariate variables that make up the state. The goal is to recover the marginal covariance on a query set $\mathcal{Q} \subseteq \mathcal{X}$ without forming the covariance matrix Σ.

Recursive covariance recovery on that Bayes-tree representation has been implemented in GTSAM for well over a decade (Dellaert, 2012; Dellaert and Kaess, 2017), but the mechanism itself has not been described previously. Throughout this report, covariance means covariance of the Gaussian posterior induced by a fixed linearization point and a fixed elimination ordering, not the covariance of the original nonlinear factor graph itself. Here we first give a mathematical account of the recursive Bayes-tree covariance mechanism that was already present for the common $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$ cases. We then build a new arbitrary-query theory on top of those existing mechanisms.

We decompose covariance recovery into query localization and block extraction. The localization problem asks which portion of the Bayes tree $\mathcal{T}$ is forced by the query set $\mathcal{Q}$. The numerical problem asks how to recover only the requested covariance blocks once that portion has been isolated. Treating these as separate tasks clarifies both the structure and the cost of the computation.

In the experimental sections below, the *pre-improvement baseline* refers specifically to the older multi-variable path used when $|\mathcal{Q}| > 2$. It does not refer to the long-standing Bayes-tree recursion already used for $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$.

The report has four parts. We first describe the existing one-variable query path already implemented in GTSAM. We then describe the existing two-variable lowest-common-ancestor query and the exact branch-summary conditionals that we call **shortcuts**. On top of those cases, we introduce one new structural contribution and one secondary numerical refinement:

1. an *exact Steiner-subtree localization* for arbitrary query sets, generalizing the existing one-path and two-path Bayes-tree queries;

2. a *solve-based covariance recovery* procedure that avoids dense inversion of the reduced information matrix once that structural reduction has been carried out.

Figure 1 previews the full workflow. We start from the full Bayes tree $\mathcal{T}$, isolate the Steiner support induced by the query set $\mathcal{Q}$, compress that support, and then recover only the covariance blocks requested on the resulting reduced system.

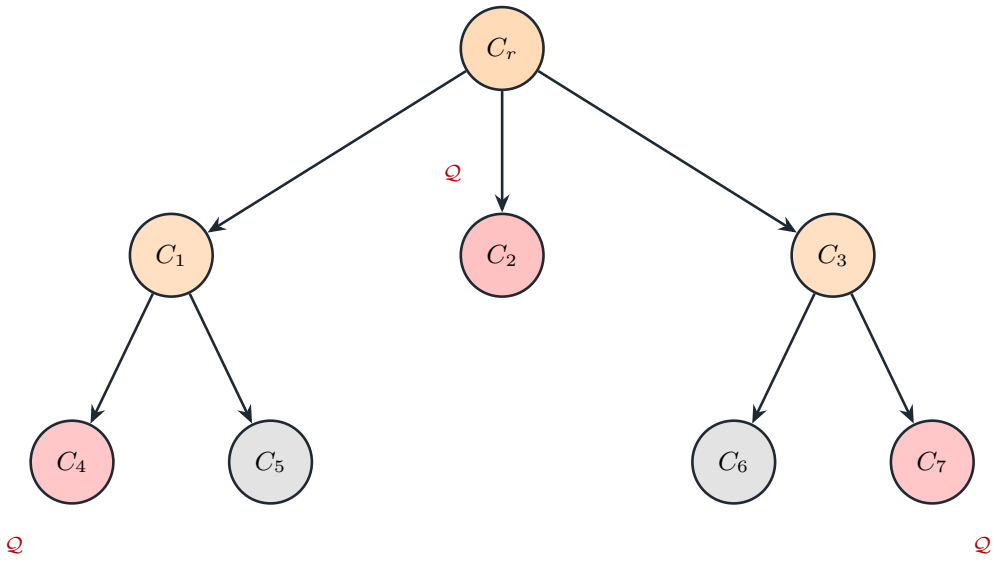

Figure 1: We study covariance queries on a full Bayes tree but compute only on the highlighted support of the query. The gray cliques belong to the full posterior, the dark red cliques contain queried variables, and the lighter highlighted cliques are the Steiner support that connects them. We then compress and eliminate only this support subtree before recovering the requested covariance.

2 Related Work

The closest prior work is selected covariance recovery from a square-root information matrix (Kaess and Dellaert, 2009). That paper studies how to recover selected marginal covariance blocks from a square-root information matrix without forming the full dense covariance matrix Σ. Its motivating application is data association, and its main technical contribution is a dynamic-programming procedure on the square-root factor that computes the particular marginal covariances needed by compatibility tests and related decision rules.

We pursue the same basic objective as in our previous work (Kaess and Dellaert, 2009). Both viewpoints seek selected covariance recovery from square-root information rather than global covariance formation. In that sense, we extend the same agenda rather than replacing it.

The main difference is that we are tree-centric rather than matrix-centric. The 2009 formulation takes the square-root matrix as the primary object and expresses recovery as a dynamic program over that factorization. Here, the Bayes tree $\mathcal{T}$ of our previous work (Dellaert and Kaess, 2017) is the primary object. This changes the structure of the problem in two ways. First, arbitrary joint-marginal queries are localized combinatorially by the Steiner subtree spanning the queried cliques, which shows that all off-subtree descendants integrate out. Second, once the query has been reduced to a smaller Gaussian system, covariance extraction is treated as a solve or selected-inversion problem on the reduced square-root factor.

Selected-inverse methods in sparse numerical linear algebra address the closest matrix-side version of our secondary numerical contribution (Erisman and Tinney, 1975; Lin et al., 2011). Those works ask how to recover specified entries or blocks of a sparse inverse from a sparse factorization without forming the full dense inverse. Our solve-based recovery is closest in spirit to that line of work, but we use it only after an exact query-local reduction on the Bayes tree $\mathcal{T}$. In other words, selected inversion explains how to recover the requested blocks once a reduced factor exists, while our main additional claim is that the correct reduced factor is itself query-local and can be extracted exactly from the clique tree $\mathcal{T}$.

The Bayes tree itself is well established in the literature (Kaess et al., 2010, 2012; Dellaert and Kaess, 2017), and GTSAM has long contained a recursive covariance-recovery mechanism built on that structure (Dellaert, 2012; Dellaert and Kaess, 2017). This report describes the existing one-variable and two-variable mechanisms explicitly and then adds a new arbitrary-query Steiner generalization on top of that existing idea, followed by a secondary solve-based extraction refinement.

Also related are sparse-precision methods in statistics and robotics (Frese, 2005; Ila et al., 2011; Zammit-Mangion and Rougier, 2018; Sidén et al., 2018). In robotics, Frese's approximate sparsity result and the mixed Kalman-information filter of Ila et al. both exploit structural locality to reduce estimation cost, but they do not formulate exact arbitrary joint-marginal recovery on a Bayes tree. In the Gaussian Markov random field literature, fast variance recovery and covariance approximation methods exploit sparse precision structure to recover selected uncertainty quantities or controlled approximations. These works reinforce the importance of locality and selected recovery, but they remain matrix-centric or approximate, whereas we give an exact clique-tree localization theorem for arbitrary joint queries and pair it with exact reduced solve-based extraction.

Our earlier work provides the structural language we use throughout (Dellaert and Kaess, 2017), and (Koller and Friedman, 2009; Blair and Peyton, 1993) make clear the older clique-tree background on both the graphical-model and sparse-linear-algebra sides. We introduced the original Bayes-tree formulation and then developed it further in incremental smoothing and mapping (Kaess et al., 2010, 2012). Our use of that representation is narrower: we use the Bayes tree $\mathcal{T}$ as the mathematical object that identifies the minimal support of a covariance query.

3 Bayes Trees and Existing Local Queries

We now formalize the recursive covariance mechanism that has long been present in the open-source GTSAM library (Dellaert, 2012; Dellaert and Kaess, 2017). This section describes the existing $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$ mechanisms before we introduce the Steiner tree generalization.

3.1 Bayes Tree Representation

A **Bayes tree** $\mathcal{T}$ is a representation of a probability distribution by rooted clique conditionals. Figure 2 shows this factorization for a Gaussian posterior $p(x)$. Let $\mathcal{T}$ be a rooted Bayes tree. Each clique $c \in \mathcal{C}(\mathcal{T})$ contains two disjoint sets of variables. For a fixed clique, we write its frontal and separator variable sets as $\mathcal{F}$ and $\mathcal{S}$:

$$\mathcal{F} \subset \mathcal{X}, \qquad \mathcal{S} \subset \mathcal{X}, \qquad \mathcal{F} \cap \mathcal{S} = \varnothing, \tag{3}$$

where $\mathcal{F}$ are the **frontals** of the clique and $\mathcal{S}$ is its **separator** with respect to its parent. The **clique conditional** is the local Gaussian factor representing one piece of the large multivariate Gaussian density $p(x)$:

$$p_c(\mathcal{F} \mid \mathcal{S}) \propto \exp\left(-\frac{1}{2}\|R_c\mathcal{F} + U_c\mathcal{S} - d_c\|^2\right). \tag{4}$$

When the clique index matters, we write $\mathcal{F}_c$ and $\mathcal{S}_c$. The full posterior $p(x)$ factorizes as

$$p(x) = \prod_{c \in \mathcal{C}(\mathcal{T})} p_c(\mathcal{F}_c \mid \mathcal{S}_c). \tag{5}$$

Each state variable appears as a frontal variable in exactly one clique, namely the clique at which that variable is eliminated. We also write

$$\mathcal{X}_c = \mathcal{F}_c \cup \mathcal{S}_c \tag{6}$$

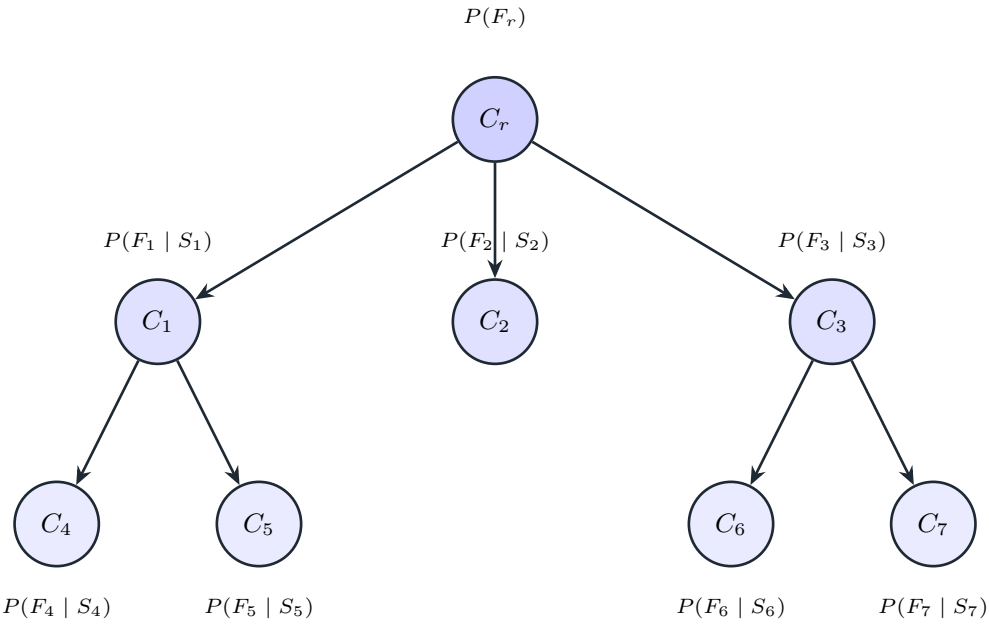

Figure 2: A Bayes tree represents the posterior as a product of clique conditionals. Each node stores a density of frontal variables conditioned on separator variables, and multiplying these local conditionals recovers the global Gaussian posterior. We use this factorization as the starting point for every covariance query.

for the variables in clique c.

3.2 Marginalization

Covariance recovery is a marginalization problem on the posterior $p(x)$. For any subset $Y \subseteq \mathcal{X}$, the marginal density $p(x_Y)$ on Y is obtained by integrating out the remaining variables:

$$p(x_Y) = \int p(x)\, dx_{\mathcal{X}\setminus Y}. \tag{7}$$

In GTSAM terms, this marginalization is carried out on the Gaussian system obtained after linearization and elimination, not on the nonlinear factor graph directly. In principle, this integral can couple all of the variables outside Y, so direct marginalization does not obviously stay local to the query. Even when the original Bayes tree is sparse, forming a marginal can destroy that sparsity and force work that reaches well beyond the queried variables. Since the basic covariance queries already used in practice are the marginal covariance of one variable and the joint covariance of two variables, the question is whether the Bayes-tree factorization can answer those queries without carrying out a global marginalization over the full state. The Bayes tree $\mathcal{T}$ does so because rooted descendant subtrees integrate to one, which keeps marginalization local.

3.3 Existing One-Variable Marginalization

We first state the existing one-variable recursion in the Bayes-tree language. Let $q \in \mathcal{X}$ be a queried variable and let $c(q)$ be the unique clique whose frontal set contains q. For that clique, write its frontal and separator sets as $\mathcal{F}$ and $\mathcal{S}$. The marginal on q is obtained from the **clique marginal**

$$p(\mathcal{F}, \mathcal{S}) = p_{c(q)}(\mathcal{F} \mid \mathcal{S})\, p(\mathcal{S}), \tag{8}$$

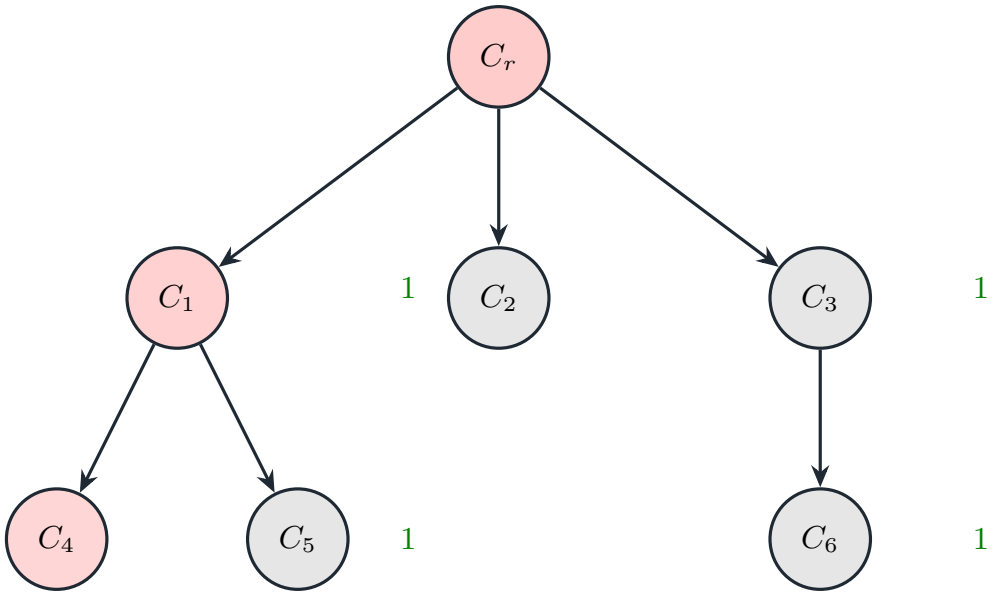

Figure 3: A descendant subtree contributes the identity after its frontal variables are integrated out. This is a property of the rooted Bayes tree itself. Once a query is fixed, that identity lets us prune any descendant branch that does not intersect the query support.

followed by marginalization of all variables except q:

$$p(x_q) = \int p_{c(q)}(\mathcal{F} \mid \mathcal{S})\, p(\mathcal{S})\, d(\mathcal{F} \cup \mathcal{S} \setminus \{q\}). \tag{9}$$

The only nonlocal term here is the **separator marginal** $p(\mathcal{S})$, and it is itself computed recursively from the parent clique. If $\mathrm{pa}(c)$ denotes the parent of clique c, and if $\mathcal{F}_c$ and $\mathcal{S}_c$ denote the frontal and separator sets of clique c, then

$$p(\mathcal{S}_c) = \int p_{\mathrm{pa}(c)}(\mathcal{F}_{\mathrm{pa}(c)} \mid \mathcal{S}_{\mathrm{pa}(c)})\, p(\mathcal{S}_{\mathrm{pa}(c)})\, d\left((\mathcal{F}_{\mathrm{pa}(c)} \cup \mathcal{S}_{\mathrm{pa}(c)}) \setminus \mathcal{S}_c\right), \tag{10}$$

with the root clique supplying the base case of an empty separator.

The reason this recursion stays local is that off-path descendant branches contribute the identity. For any clique c, let the **descendant subtree** $\mathcal{T}_c$ denote the rooted subtree consisting of c and all of its descendants. More generally, for any rooted subtree $\mathcal{T}'$, let

$$\mathcal{X}_{\mathcal{T}'} = \bigcup_{d \in \mathcal{C}(\mathcal{T}')} \mathcal{F}_d \tag{11}$$

be the variables eliminated in that subtree. Since separators belong to ancestors, $\mathcal{S}_c$ lies outside $\mathcal{X}_{\mathcal{T}_c}$. Applying marginalization to the whole subtree gives

$$m_c(\mathcal{S}_c) = \int \prod_{d \in \mathcal{C}(\mathcal{T}_c)} p_d(\mathcal{F}_d \mid \mathcal{S}_d)\, d\mathcal{X}_{\mathcal{T}_c}, \tag{12}$$

and by normalization of conditional densities this integral is identically one:

$$m_c(\mathcal{S}_c) \equiv 1. \tag{13}$$

Figure 3 shows this pruning identity geometrically. Therefore every descendant branch off the rootward path from $c(q)$ can be replaced by the identity, so only the clique containing q and its ancestors influence the marginal on q. In other words, the single-variable query reduces to one rootward path together with recursive separator marginals supplied from above. The single-variable query is thus the degenerate one-path case of Bayes-tree covariance recovery already present in GTSAM.

3.4 Existing Two-Variable Query via LCA Shortcuts

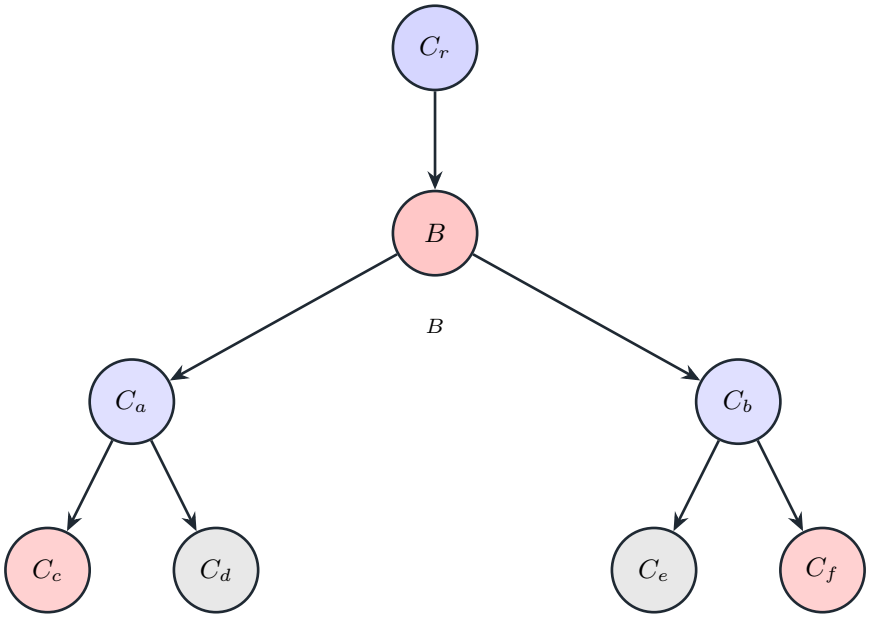

Figure 4: The two-variable Bayes-tree query is supported on the two rootward paths to the lowest common ancestor. For a pair query, the relevant cliques are the ones on those two paths, together with their meeting point at the ancestor clique B.

The two-variable case adds one branching point but remains an existing localized Bayes-tree query. Consider the two-variable query set $\mathcal{Q} = \{q_1, q_2\}$, and let

$$c_1 = c(q_1), \qquad c_2 = c(q_2) \tag{14}$$

be the cliques containing the queried variables. Let b be the lowest common ancestor of c_1 and c_2. Then the exact support of the pair query is the union of the two rootward paths from c_1 and c_2 to b. Figure 4 shows this pairwise support explicitly.

The ancestor clique b contributes its clique marginal

$$p(\mathcal{F}_b, \mathcal{S}_b) = p_b(\mathcal{F}_b \mid \mathcal{S}_b)\, p(\mathcal{S}_b). \tag{15}$$

Each of the two descendant-to-ancestor branches contributes an exact conditional summary of the queried-clique separator on the ancestor clique variables. This summary was already implemented in GTSAM, and we refer to it here as a **path shortcut**. The queried clique conditional itself is kept separate and is added explicitly later.

To define that shortcut, let u be one of the queried cliques c_1 or c_2, and let

$$\pi(u, b) = (d_0 = u, d_1, \ldots, d_\ell = b) \tag{16}$$

be the unique path from u to the ancestor b, with no branching clique on that path except possibly u and b. Along this path, the shortcut keeps two interface sets:

$$\mathcal{A}_{u,b} = \mathcal{X}_b \cap \left(\bigcup_{j=0}^{\ell-1} \mathcal{S}_{d_j} \right), \tag{17}$$

$$\mathcal{B}_{u,b} = \mathcal{S}_u \setminus \mathcal{X}_b. \tag{18}$$

The set $\mathcal{A}_{u,b}$ is the **ancestor-side interface**, namely the branch separator variables already present in clique b. The set $\mathcal{B}_{u,b}$ is the **descendant-side interface**, namely the queried-clique separator variables that must still be represented below the ancestor after the strict path is compressed. Let

$$\mathcal{Z}_{u,b} = \left(\bigcup_{j=1}^{\ell-1} \mathcal{X}_{d_j} \right) \setminus (\mathcal{A}_{u,b} \cup \mathcal{B}_{u,b}) \tag{19}$$

collect the **internal path variables** that are integrated out. The path shortcut is then

$$\chi_{u\to b}(\mathcal{B}_{u,b} \mid \mathcal{A}_{u,b}) = \int \prod_{j=1}^{\ell-1} p_{d_j}(\mathcal{F}_{d_j} \mid \mathcal{S}_{d_j})\, d\mathcal{Z}_{u,b}. \tag{20}$$

This shortcut is therefore an exact Gaussian conditional obtained by eliminating the strictly internal path variables while retaining only the ancestor-side and descendant-side interfaces. When $\mathcal{B}_{u,b} = \emptyset$, the shortcut is trivial and contributes no factor. The queried-clique conditional $p_u(\mathcal{F}_u \mid \mathcal{S}_u)$ is not part of the shortcut and must be included explicitly in the final pairwise assembly.

For the two-variable query $\mathcal{Q} = \{q_1, q_2\}$, the joint density is therefore assembled from the ancestor clique marginal $p(\mathcal{F}_b, \mathcal{S}_b)$, the two queried-clique conditionals $p_{c_1}(\mathcal{F}_{c_1} \mid \mathcal{S}_{c_1})$ and $p_{c_2}(\mathcal{F}_{c_2} \mid \mathcal{S}_{c_2})$, and the two path shortcuts $\chi_{c_1\to b}$ and $\chi_{c_2\to b}$. When neither queried clique already equals b, the pair marginal can be written as

$$\begin{aligned} p(q_1, q_2) = \int & p(\mathcal{F}_b, \mathcal{S}_b)\, \chi_{c_1\to b}(\mathcal{B}_{c_1,b} \mid \mathcal{A}_{c_1,b})\, \chi_{c_2\to b}(\mathcal{B}_{c_2,b} \mid \mathcal{A}_{c_2,b}) \\ & p_{c_1}(\mathcal{F}_{c_1} \mid \mathcal{S}_{c_1})\, p_{c_2}(\mathcal{F}_{c_2} \mid \mathcal{S}_{c_2}) \\ & d\left((\mathcal{X}_b \cup \mathcal{B}_{c_1,b} \cup \mathcal{B}_{c_2,b} \cup \mathcal{F}_{c_1} \cup \mathcal{F}_{c_2}) \setminus \{q_1, q_2\} \right). \end{aligned} \tag{21}$$

This is the existing two-variable Bayes-tree marginalization in its localized form: the ancestor clique contributes the shared coupling, each descendant branch contributes one queried-clique conditional together with at most one shortcut, and everything else on the support is integrated out. The single-variable case above is the degenerate version with one branch, while the present two-variable case is the existing two-branch path-union case. If one queried clique already equals the ancestor clique b, then its conditional is already contained in $p(\mathcal{F}_b, \mathcal{S}_b)$, and the corresponding shortcut and extra queried-clique factor are omitted.

Figure 5 shows these sets for an example with query $\mathcal{Q} = \{x_1, x_7\}$. In that test, $c_1 = C_3$, $c_2 = C_4$, and the lowest common ancestor is $b = C_1$ with $\mathcal{X}_b = \{x_5, x_6, x_4\}$. On the left branch, $\mathcal{A}_{C_3,C_1} = \{x_4\}$, $\mathcal{B}_{C_3,C_1} = \{x_2\}$, and $\mathcal{Z}_{C_3,C_1} = \{x_3\}$, so the shortcut is $\chi_{C_3\to C_1}(x_2 \mid x_4)$ and the queried-clique factor is $p_{C_3}(x_1 \mid x_2)$. On the right branch, $\mathcal{A}_{C_4,C_1} = \{x_6\}$, $\mathcal{B}_{C_4,C_1} = \emptyset$, and $\mathcal{Z}_{C_4,C_1} = \emptyset$, so the shortcut is trivial and the branch contributes only $p_{C_4}(x_7 \mid x_6)$.

The same balanced tree also exhibits the ancestor-clique edge case. For the query $\mathcal{Q} = \{x_4, x_1\}$, the clique $c_1 = C_1$ is already the ancestor of $c_2 = C_3$, so $b = C_1$. In that case the x_4 branch contributes only through the ancestor clique marginal $p(\mathcal{F}_{C_1}, \mathcal{S}_{C_1})$: there is no nontrivial shortcut $\chi_{C_1\to C_1}$ and no extra queried-clique factor for C_1. The only nontrivial descendant branch is still the shortcut $\chi_{C_3\to C_1}(x_2 \mid x_4)$ together with the queried-clique conditional $p_{C_3}(x_1 \mid x_2)$.

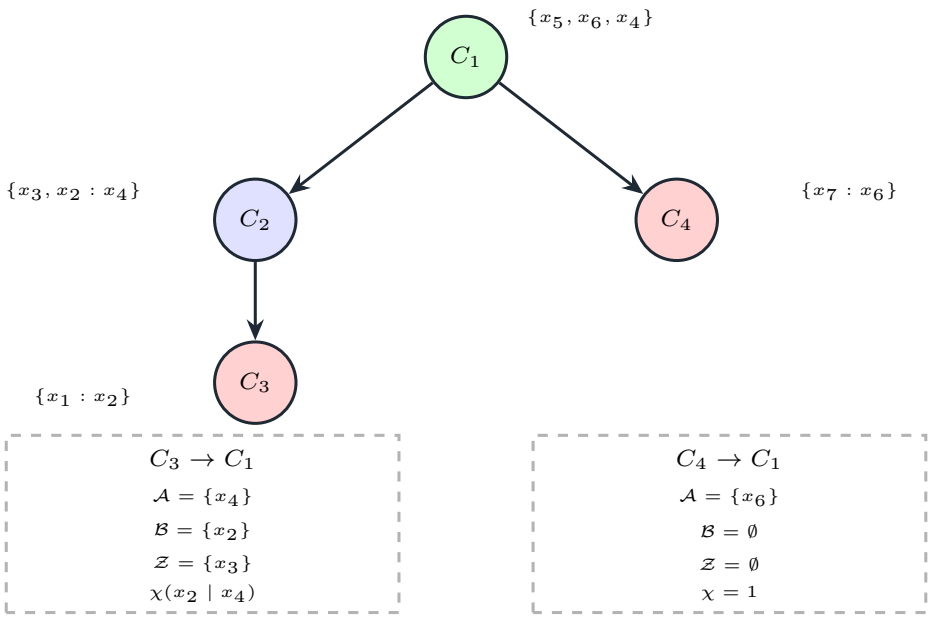

Figure 5: Worked two-variable example. The lower branch panels list the interface sets and shortcut for each rootward path to the ancestor clique C_1.

4 Arbitrary Query Support via Steiner Subtrees

The two-variable case already identifies the support as the union of the two rootward paths to the lowest common ancestor. For larger query sets $\mathcal{Q}$, the natural extension is the minimal connected subtree spanning all queried cliques. Every query set $\mathcal{Q}$ induces a unique minimal supporting subtree. Let $\mathcal{Q} \subseteq \mathcal{X}$ be a nonempty query set. For each query variable $q \in \mathcal{Q}$, let $c(q)$ be the unique clique whose frontal set contains q. Define the **support cliques**

$$\mathcal{C}_{\mathcal{Q}} = \{c(q) : q \in \mathcal{Q}\}. \tag{22}$$

Definition 1 (Steiner subtree)**.** The **Steiner subtree** $\mathcal{T}_{\mathcal{Q}} \subseteq \mathcal{T}$ is the minimal connected subtree whose clique set contains $\mathcal{C}_{\mathcal{Q}}$.

Because the ambient graph is already a tree, the Steiner subtree $\mathcal{T}_{\mathcal{Q}}$ is just the union of rootward paths from the support cliques $\mathcal{C}_{\mathcal{Q}}$. Figure 6 shows this minimal support. Since $\mathcal{T}$ is a tree, $\mathcal{T}_{\mathcal{Q}}$ is simply the union of the rootward paths from all cliques in $\mathcal{C}_{\mathcal{Q}}$ to their common branching structure.

This definition is the exact extension of the earlier one-variable path case and the two-variable lowest-common-ancestor case to arbitrary query sets.

4.1 Localization Principle

The marginal $p(x_{\mathcal{Q}})$ on a query set $\mathcal{Q}$ depends only on the cliques in its Steiner subtree $\mathcal{T}_{\mathcal{Q}}$.

Theorem 1 (Localization to the Steiner subtree)**.** *Let $\mathcal{Q} \subseteq \mathcal{X}$ be a query set and let $\mathcal{T}_{\mathcal{Q}}$ be the*

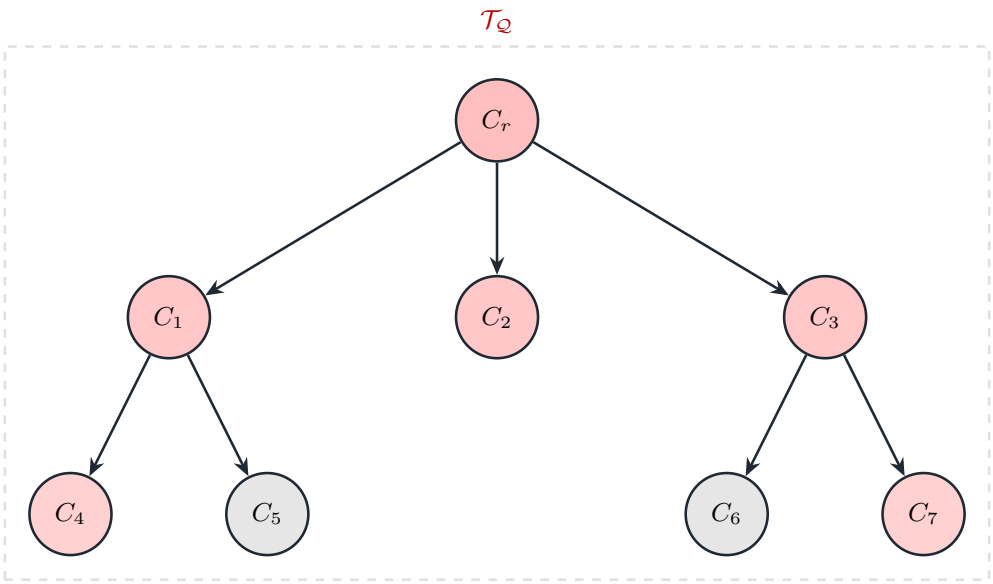

Figure 6: The Steiner subtree is the exact support of a joint-marginal query. The highlighted cliques are the minimal connected subtree spanning the query cliques, and every gray branch outside that subtree integrates out by the pruning identity. The localization theorem states that the query marginal depends only on this highlighted structure.

corresponding Steiner subtree. Then the marginal density on $\mathcal{Q}$ *can be written as*

$$p(x_\mathcal{Q}) = \int \prod_{c \in \mathcal{C}(\mathcal{T}_\mathcal{Q})} p_c(\mathcal{F}_c \mid \mathcal{S}_c)\, d(\mathcal{X}_{\mathcal{T}_\mathcal{Q}} \setminus \mathcal{Q}). \tag{23}$$

All cliques outside $\mathcal{T}_\mathcal{Q}$ *integrate out to one.*

The proof is a repeated pruning argument on off-subtree descendants.

Proof. Consider any child subtree $\mathcal{T}_u$ attached to a clique $b \in \mathcal{T}_\mathcal{Q}$ such that no clique of $\mathcal{T}_u$ belongs to $\mathcal{C}_\mathcal{Q}$. By definition of $\mathcal{T}_\mathcal{Q}$, every variable in $\mathcal{Q}$ lies outside $\mathcal{X}_{\mathcal{T}_u}$. Hence

$$\int \prod_{c \in \mathcal{C}(\mathcal{T}_u)} p_c(\mathcal{F}_c \mid \mathcal{S}_c)\, d\mathcal{X}_{\mathcal{T}_u} = 1. \tag{24}$$

Applying this elimination to every maximal off-subtree descendant yields the claim. □

This theorem is the first new structural step beyond the existing one-variable and two-variable mechanisms. It identifies the exact support of an arbitrary joint query. Operationally, this is the step that extends the long-standing localized $|\mathcal{Q}| = 2$ path to arbitrary $|\mathcal{Q}|$, replacing the older multi-query fallback that re-eliminated the Gaussian system globally.

4.2 Generalizing Existing Shortcuts to Steiner Compression

Once the support has been reduced from the full tree $\mathcal{T}$ to the Steiner subtree $\mathcal{T}_\mathcal{Q}$, the existing pairwise shortcut idea extends directly to arbitrary Steiner trees. The next step is to compress internal chains of cliques that do not themselves contain query variables and are not branch points.

The path-shortcut definition above already summarizes a non-branching chain by a single conditional over its interface to the ancestor clique. On a Steiner subtree, we apply that same construction to every maximal non-branching chain. The exact interface partition is less important than the fact that the whole chain has been replaced by one exact shortcut. What matters is that the path is compressed into a conditional over the interface it presents to the ancestor clique.

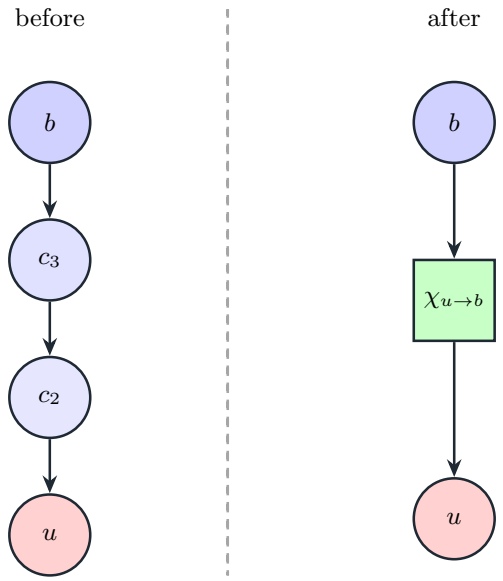

Figure 7: Path compression replaces a nonbranching clique chain by one shortcut conditional. The chain on the left and the compressed connection on the right induce the same query marginal, but the compressed form has fewer internal cliques and therefore a smaller reduced problem. This is the generalized form of the shortcut idea already used in the pairwise Bayes-tree query.

After path compression, the query set $\mathcal{Q}$ is supported on a reduced tree containing only structural necessities. Figure 7 shows how a whole nonbranching chain is replaced by a single shortcut $\chi_{u\to b}$. After compressing all maximal nonbranching chains, one obtains a **reduced query tree** $\widehat{\mathcal{T}}_{\mathcal{Q}}$ whose clique set contains only:

1. cliques carrying query variables,
2. branch cliques required to connect them,
3. the lowest common ancestral structure needed by the query.

The reduced tree $\widehat{\mathcal{T}}_{\mathcal{Q}}$ gives an exact factorization of the query marginal $p(x_{\mathcal{Q}})$. The marginal on $\mathcal{Q}$ is then

$$p(x_{\mathcal{Q}}) = \int \left(\prod_{b\in\mathcal{C}(\widehat{\mathcal{T}}_{\mathcal{Q}})} \varphi_b \right) \left(\prod_{(u\to b)\in E(\widehat{\mathcal{T}}_{\mathcal{Q}})} \chi_{u\to b} \right) d(\mathcal{X}_{\widehat{\mathcal{T}}_{\mathcal{Q}}} \setminus \mathcal{Q}), \tag{25}$$

where φ_b denotes the local clique conditional or clique marginal retained at node b.

The computational benefit is that query cost now scales with the Steiner subtree $\mathcal{T}_{\mathcal{Q}}$ rather than the full problem. Let $\omega(\mathcal{T}_{\mathcal{Q}})$ denote the maximum frontal-plus-separator block size encountered inside the Steiner subtree $\mathcal{T}_{\mathcal{Q}}$. Then the query reduction cost scales with the geometry of $\mathcal{T}_{\mathcal{Q}}$, not with the size of the full tree $\mathcal{T}$. If the query set $\mathcal{Q}$ is small and spatially localized, then $\mathcal{T}_{\mathcal{Q}}$ is typically much smaller than $\mathcal{T}$.

The main practical consequence is that overlapping queries can reuse the same compressed structural support. This improves on any method that re-eliminates the full graph for every multi-variable query. The gain is especially pronounced when many queries overlap, because the generalized branch and path shortcuts may be reused across related query sets.

5 Secondary Numerical Improvement: Covariance Recovery without Dense Inversion

After the existing localized Bayes-tree recursion has been generalized to arbitrary queries through Steiner support reduction, a separate numerical question remains: how should covariance blocks be extracted from the reduced system? Our secondary contribution is to replace dense inversion on that reduced problem by solves on the reduced square-root factor $R_{\mathcal{Q}}$. Assume the structural reduction above has produced an exact reduced Gaussian system $p(x_{\mathcal{Q}})$ on the query variables:

$$p(x_{\mathcal{Q}}) \propto \exp\left(-\frac{1}{2}\|R_{\mathcal{Q}}x_{\mathcal{Q}} - d_{\mathcal{Q}}\|^2\right), \tag{26}$$

with

$$\Lambda_{\mathcal{Q}} = R_{\mathcal{Q}}^\top R_{\mathcal{Q}}, \qquad \Sigma_{\mathcal{Q}} = \Lambda_{\mathcal{Q}}^{-1}. \tag{27}$$

The conventional approach forms the reduced information matrix $\Lambda_{\mathcal{Q}}$ and computes the reduced covariance matrix $\Sigma_{\mathcal{Q}}$ by dense inversion. We replace this by triangular solves or selected inversion.

Selected covariance columns are solutions of a reduced linear system. Let $E \in \mathbb{R}^{|\mathcal{Q}|\times k}$ be a block column selector whose columns are standard basis vectors corresponding to the variables or blocks of interest. The selected covariance columns

$$\Sigma_{\mathcal{Q}}E \tag{28}$$

solve

$$\Lambda_{\mathcal{Q}}X = E. \tag{29}$$

Because $\Lambda_{\mathcal{Q}} = R_{\mathcal{Q}}^\top R_{\mathcal{Q}}$, this is equivalent to the pair of triangular systems

$$R_{\mathcal{Q}}^\top Y = E, \tag{30}$$

$$R_{\mathcal{Q}}X = Y. \tag{31}$$

Hence

$$X = \Sigma_{\mathcal{Q}}E. \tag{32}$$

This formulation computes exactly the columns that the query asks for. Diagonal blocks are obtained by selecting the basis vectors associated with those blocks. Cross-covariance blocks between two variable sets I and J are obtained by selecting $E = E_J$ and reading rows I from X.

When many overlapping blocks are needed, selected inversion is the natural block generalization of repeated solves. Consider a block partition

$$R_{\mathcal{Q}} = \begin{bmatrix} R_{11} & R_{12} \\ 0 & R_{22} \end{bmatrix}, \tag{33}$$

induced by the elimination order. Then

$$\Sigma_{\mathcal{Q}} = \begin{bmatrix} R_{11}^{-1}R_{11}^{-\top} + R_{11}^{-1}R_{12}\Sigma_{22}R_{12}^\top R_{11}^{-\top} & -R_{11}^{-1}R_{12}\Sigma_{22} \\ -\Sigma_{22}R_{12}^\top R_{11}^{-\top} & \Sigma_{22} \end{bmatrix}, \tag{34}$$

where

$$\Sigma_{22} = (R_{22}^{\top} R_{22})^{-1}. \tag{35}$$

This identity yields a backward recursion from leaves to root that computes exactly those blocks of $\Sigma_{\mathcal{Q}}$ that are coupled by the reduced elimination structure.

The selected-inversion viewpoint matters because most queries do not ask for the full dense covariance matrix $\Sigma_{\mathcal{Q}}$ on the reduced state $x_{\mathcal{Q}}$. What is required is a specified set of covariance blocks, and those blocks can be propagated through the reduced factorization directly.

The numerical consequence is that dense inversion becomes unnecessary even after structural reduction. The solve-based approach has three advantages:

1. it avoids dense inversion, which is numerically less attractive than triangular solves;
2. it preserves sparsity longer by operating on $R_{\mathcal{Q}}$ rather than on a dense Hessian;
3. it permits partial covariance extraction when the user needs only diagonal or a few off-diagonal blocks.

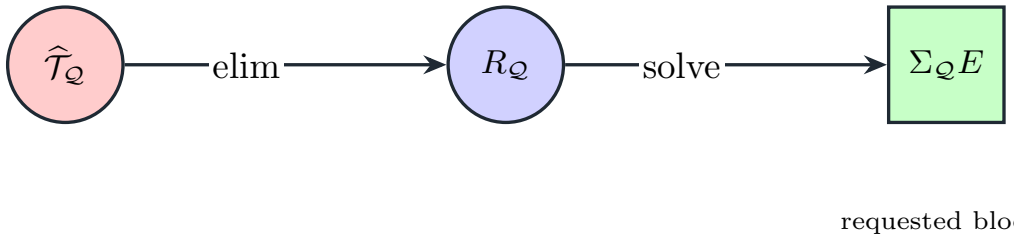

Figure 8: Solve-based extraction separates structural reduction from numerical recovery. After the reduced query tree has been eliminated to a reduced square-root factor $R_{\mathcal{Q}}$, the method does not form a dense inverse of $\Lambda_{\mathcal{Q}} = R_{\mathcal{Q}}^{\top} R_{\mathcal{Q}}$. Instead, triangular solves or selected inversion recover exactly the covariance columns and blocks requested by the query.

Figure 8 summarizes this second stage. Once the query set $\mathcal{Q}$ has been reduced structurally, covariance recovery proceeds by triangular solves or selected inversion on the reduced square-root factor $R_{\mathcal{Q}}$ rather than by dense inversion.

6 Combined Query Algorithm

The existing local recursion, the new Steiner generalization, and the secondary solve-based extraction step combine into a two-stage query procedure.

Stage 1: combinatorial localization

The first stage isolates the exact reduced structure needed by the query set $\mathcal{Q}$. Given $\mathcal{Q}$:

1. identify the support cliques $\mathcal{C}_{\mathcal{Q}}$;
2. form the Steiner subtree $\mathcal{T}_{\mathcal{Q}}$;
3. prune all off-subtree descendants using the localization theorem;
4. compress maximal nonbranching paths into path shortcuts;
5. assemble the reduced query tree $\widehat{\mathcal{T}}_{\mathcal{Q}}$ and eliminate internal variables not in $\mathcal{Q}$.

Stage 2: numerical covariance extraction

The second stage extracts only the covariance entries that the query set $\mathcal{Q}$ requests. Given the reduced square-root factor $R_{\mathcal{Q}}$:

1. if the full covariance on $\mathcal{Q}$ is needed, use block selected inversion on $R_{\mathcal{Q}}$;
2. if only specified blocks are needed, solve

$$R_{\mathcal{Q}}^{\top} Y = E, \qquad R_{\mathcal{Q}} X = Y, \tag{36}$$

   and extract the requested rows and columns from X.

The result is that structure and numerics are now separated cleanly. This separates the structural question, "which part of the tree matters," from the numerical question, "which covariance entries are needed."

7 Complexity Discussion

The complexity is governed by the reduced query geometry and by the number of requested covariance columns, not by the size of the full state. Let N be the size of the full problem, let M be the size of the reduced query system after Steiner compression, and let k be the number of requested covariance columns.

- A global dense inversion has cost cubic in N and destroys sparsity.
- Re-eliminating the full graph for each query retains sparse elimination but still pays a global combinatorial cost.
- The structural contribution reduces elimination to the query-local size M by localizing arbitrary queries to their Steiner support and compressing that support by generalized shortcuts.
- The secondary numerical improvement replaces dense inversion on the reduced problem by either:
  1. block selected inversion on the reduced factorization, or
  2. two triangular solves per requested covariance column.

The consequence is a query-local cost model. Thus the overall cost is governed by the size and width of the Steiner subtree $\mathcal{T}_{\mathcal{Q}}$ and by the number of requested covariance blocks, rather than by the total state dimension.

8 Experimental Results

We benchmarked the implemented algorithms on the simulated Manhattan pose-graph datasets `w100`, `w10000`, and `w20000` introduced by Olson et al. (2006) and used again in previous work (Kaess et al., 2010). For each dataset, we anchored the first pose with a tight prior to remove gauge freedom, optimized once to a reference estimate, linearized once at that estimate, and then timed only covariance queries on the resulting Gaussian system. We evaluated local windows, wide-separated query sets, and overlapping windows under both COLAMD and METIS orderings. For the local single-pose, pair-pose, and contiguous-window families, the sampled starts were spaced

evenly across the full valid trajectory range rather than drawn from only the earliest windows. Each distinct query was run once as an untimed warmup and then 10 times as measured repetitions. The timing summaries below report the median across the sampled queries in each bucket, where each query contributes its mean time over those repeated runs. The structural summaries use the same median-across-queries aggregation. Here $|\mathcal{Q}|$ counts `Pose2` variables, whereas the reduced-state dimension later counts scalar degrees of freedom in the reduced Gaussian system.

Overview

- The historical recursive Bayes-tree mechanism already handled the common $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$ cases locally in GTSAM, which is why the gains there are modest; see Section 8.1.
- The large speedups begin once $|\mathcal{Q}| > 2$, where the new Steiner generalization replaces the older multi-key fallback to global re-elimination of the linearized Gaussian system; see Section 8.2.
- Ordering matters because it changes the size and width of the Steiner support, not because it changes the cost of a fixed dense linear-algebra kernel; see Section 8.3.
- Support geometry is the right predictor of covariance-query cost, but clique counts alone are not enough: separator width explains the remaining dataset-to-dataset differences; see Section 8.4.
- The striking `w10000` versus `w20000` gap comes from different local support inflation and separator widths along the sampled windows, not from the larger graph being intrinsically easier; see Section 8.5.
- In the new sequential `ISAM2` experiment on `w20000`, update cost spikes much more than the pairwise covariance query cost, while the $\{x_0, x_t\}$ query itself remains in a stable few-millisecond band after the path is established; see Section 8.6.

8.1 Baseline Queries: $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$

We begin with the query sizes that were already optimized by the existing recursive Bayes-tree mechanism before the present Steiner generalization. Figure 9 shows that single-pose and pair-pose queries stay in the same sub-millisecond regime across all four variants. The lower absolute times on `w20000` should not be read as a graph-size effect: for these already-local queries, the cost is governed mainly by the local Bayes-tree/clique geometry around the sampled poses, not by the total number of poses in the dataset. For $|\mathcal{Q}| = 1$, where all methods reduce to the same marginal-factor computation, replacing dense inversion by solve-based recovery mainly affects the extraction term rather than the total time: on `w10000` with COLAMD the extraction step drops from about 0.000300 ms to about 0.000158 ms, while the total time moves only from about 0.0241 ms to about 0.0207 ms. On `w20000` with COLAMD, the extraction term similarly drops from about 0.000300 ms to about 0.000148 ms, while the total time stays near 0.0078–0.0080 ms. These changes are necessarily small because the extracted matrices are already only 3×3.

The pairwise case confirms that the generalized method does not penalize the older common Bayes-tree query. For $|\mathcal{Q}| = 2$, all variants again remain tightly clustered: for example, on `w10000` with COLAMD they range only from about 0.1006 ms to about 0.1096 ms, and on `w20000` with METIS they range only from about 0.0279 ms to about 0.0314 ms. This is the right baseline for the later results. The large gains do not start at $|\mathcal{Q}| = 1$ or $|\mathcal{Q}| = 2$ because those queries were

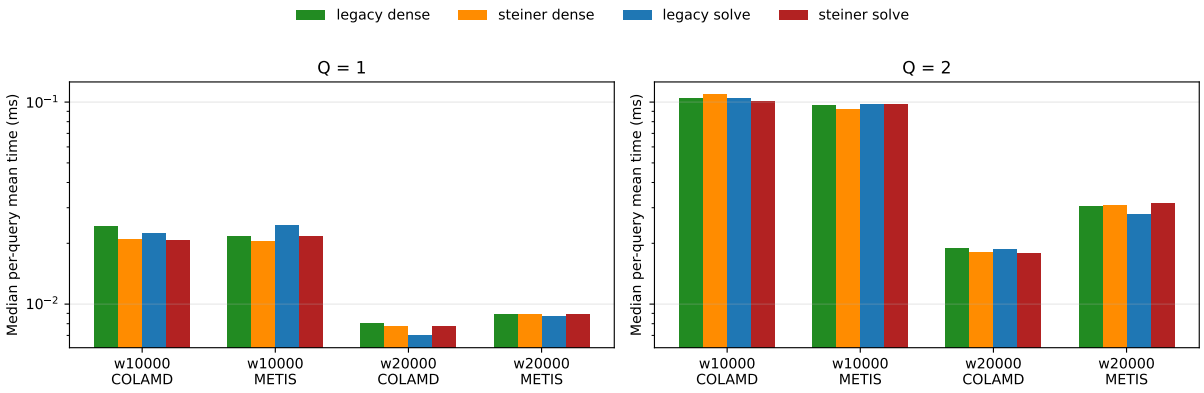

Figure 9: Baseline common queries remain in the same performance regime. For $|\mathcal{Q}| = 1$ and $|\mathcal{Q}| = 2$ on `w10000` and `w20000`, all four variants cluster tightly on the log scale because these cases were already local in the pre-improvement baseline. The absolute difference between the two datasets reflects local Bayes-tree geometry near the sampled poses rather than the total graph size. Solve-based extraction yields only modest gains here, which is exactly why the much larger speedups in the later figures should be read as a correction to the older $|\mathcal{Q}| > 2$ path rather than as a blanket acceleration of every covariance query.

already handled by the recursive Bayes-tree mechanism. They start exactly where the older multi-key baseline left that mechanism and reverted to global re-elimination of the linearized Gaussian system.

8.2 Local Joint Queries

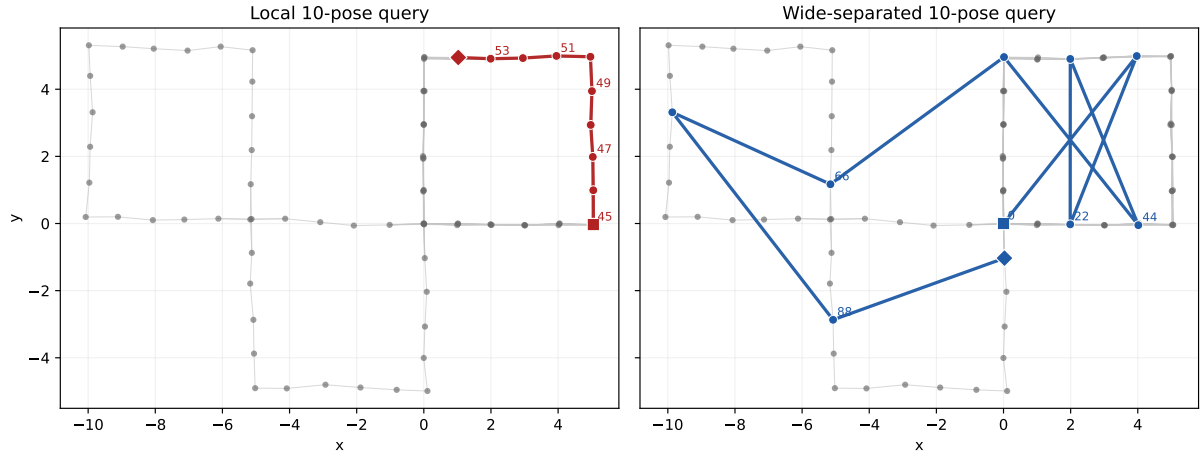

Figure 10: The `w100` pose graph makes the benchmark query families concrete. The left panel highlights a contiguous 10-pose local window, while the right panel highlights a 10-pose wide-separated query spread across the map. These two query shapes induce very different Steiner supports and therefore very different covariance-query costs.

We now look at larger queries with $|\mathcal{Q}| > 2$. Figure 10 shows a small Manhattan-world pose graph with 100 poses, together with two representative 10-pose queries from the benchmark families: a contiguous local window near the center of the trajectory and a wide-separated query that spans

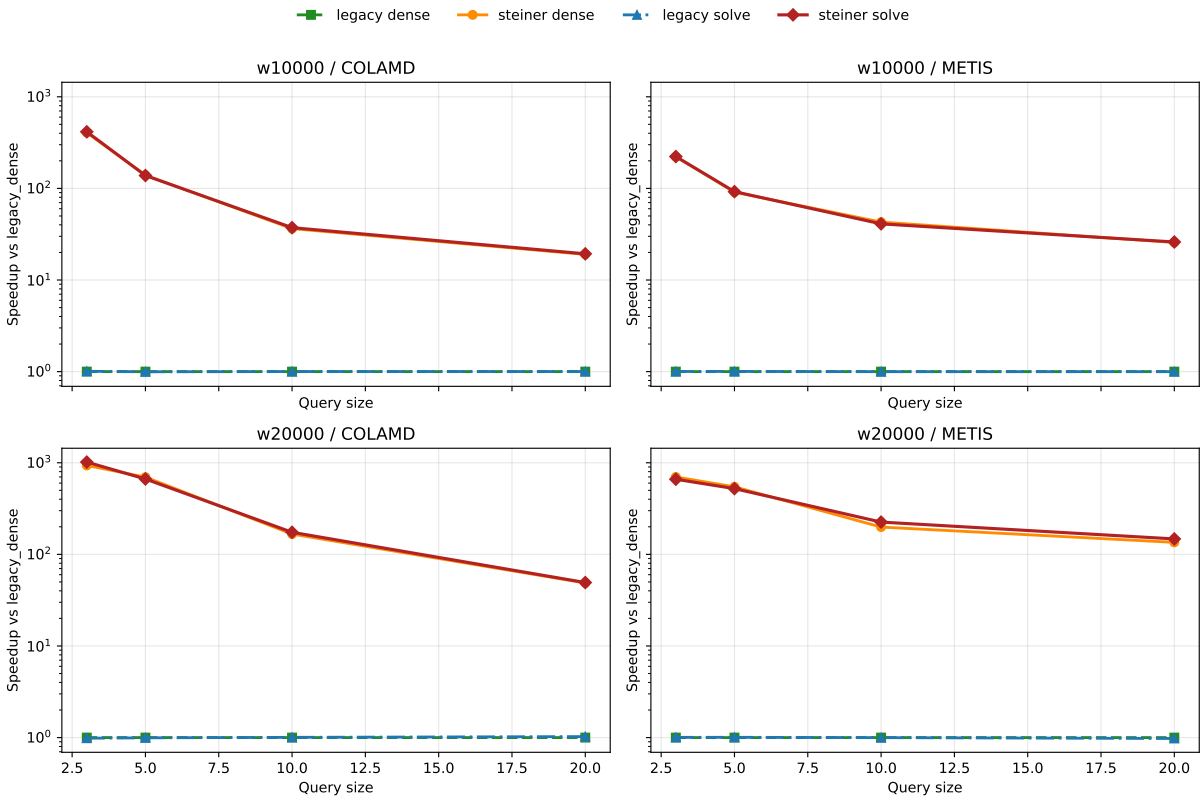

Figure 11: Steiner localization removes most of the cost for local joint covariances. For local windows on `w10000` and `w20000`, both Steiner-based variants are much faster than both pre-improvement variants, while the dense-versus-solve choice only changes the remaining small extraction cost. This figure shows that the new Steiner generalization is the main reason the method improves on the earlier multi-key baseline for larger joint queries.

the full map. These are small enough to visualize directly but already capture the two extreme support geometries that drive the larger timing trends.

Local window queries show that the Steiner generalization removes the global re-elimination cliff. Figure 11 shows that on `w10000` with COLAMD, the combined Steiner-plus-solve method is about 37× faster than the pre-improvement dense baseline for 10-pose local windows and about 19× faster for 20-pose windows. On `w20000`, the same comparison yields about 174× and 49× speedups. These gains occur because local windows keep the support close to the query itself, so the localized computation avoids the tens of milliseconds spent by the full re-elimination fallback used by the earlier multi-key path. Within each Steiner-localized variant, replacing dense inversion by solve-based extraction yields a smaller additional gain by reducing only the remaining recovery step on the reduced system.

8.3 Ordering Sensitivity

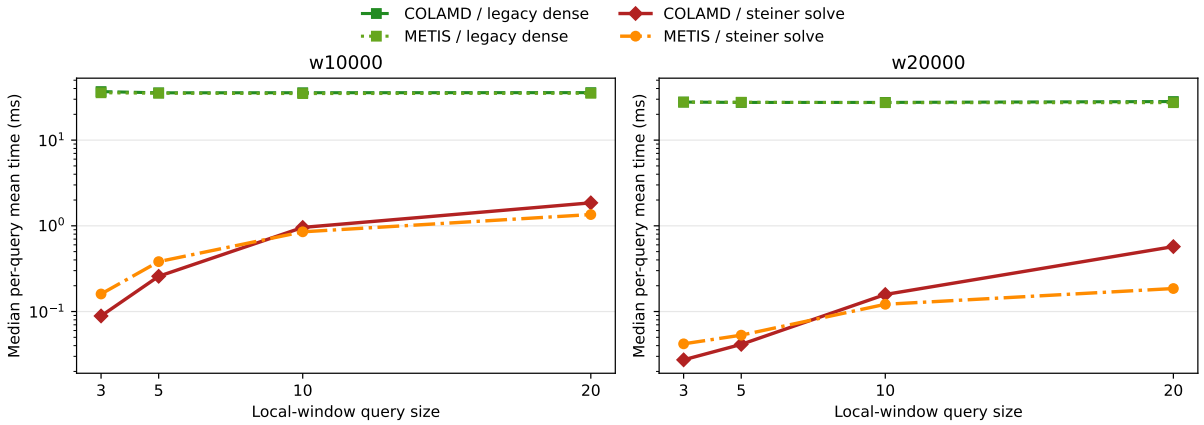

Figure 12: Ordering changes query cost through support geometry, and the dense pre-improvement baseline shows that this effect is specific to the localized method. The plotted runtimes compare the pre-improvement dense variant and Steiner solve for local-window joint covariances under COLAMD and METIS. The dense baseline is nearly flat across orderings, whereas the localized method reflects how each ordering reshapes the support subtree.

Ordering matters because it changes the geometry of the Steiner support rather than just the cost of a fixed numerical kernel. Figure 12 shows that the dense pre-improvement baseline stays in the same tens-of-milliseconds range under both COLAMD and METIS, whereas the localized method moves with the support geometry induced by the ordering. For the local-window queries shown here, both orderings keep the support close to the query size, so the localized method is cheap under either ordering. On `w10000`, a 20-pose local window drops from about 1.85 ms under COLAMD to about 1.35 ms under METIS even though the median support changes little (24 cliques versus 25.5). The reason is width: the median maximum separator dimension drops from about 115.5 to about 94.5. On `w20000`, the same comparison drops from about 0.572 ms to about 0.186 ms as the median support shrinks from 35 to 24 cliques and the median maximum separator dimension drops from about 18 to about 16.5. The same structural principle becomes more consequential on less local queries, where both support size and support width can vary more strongly with the ordering.

8.4 Support Geometry Predicts Query Cost

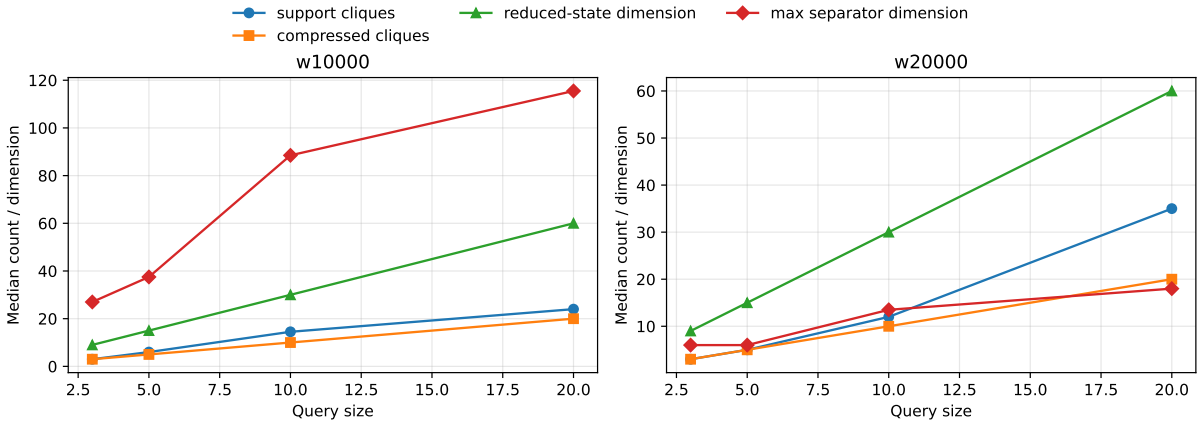

Figure 13: Support geometry predicts localized query cost. Local windows keep the support close to the query size, whereas larger or less local queries inflate the support and only then require substantial work. The plotted reduced-state dimension is the total variable dimension of the localized Gaussian system after support extraction and path compression. The maximum separator dimension measures the widest separator encountered on the localized support and captures width effects that clique counts alone miss. All structural curves are medians across the sampled queries in each benchmark bucket. The timing curves in Figures 11 and 12 follow these support and width statistics closely.

Support statistics explain when localization helps and when it can still be expensive. Figure 13 shows this relationship directly. For local windows, the support clique count, the compressed clique count, and the reduced-state dimension all stay in the same rough range as the query size, which is why local queries become cheap. Here the reduced-state dimension means the total scalar dimension of the reduced Gaussian system that remains after pruning away off-support branches and compressing internal paths, so a 20-pose query starts at 60 scalar degrees of freedom before any extra branch or separator variables are retained. For wide-separated queries, the support can expand far beyond the number of queried variables, especially under COLAMD on the larger grids. The maximum separator dimension is the complementary width metric: it explains why two query families with similar support counts can still differ substantially in runtime. Support geometry therefore means both how much of the Bayes tree is touched and how wide that touched portion becomes.

8.5 Why `w20000` Can Be Faster Than `w10000`

Figure 14 resolves the most surprising batch result. The earlier benchmark over-sampled only the first windows in each dataset. After resampling local windows evenly across the full trajectory, `w20000` is still often faster than `w10000`, but now the reason is visible in the support geometry. For 10-pose windows under COLAMD, `w20000` reaches exact support ($|\mathcal{T}_\mathcal{Q}| = |\mathcal{Q}|$) on 7 of the 16 sampled windows, whereas `w10000` does so on only 2. The median support ratio is therefore about 1.2 for `w20000` and about 1.45 for `w10000`. Width matters even more strongly: the median maximum separator dimension is only about 13.5 on `w20000` but about 88.5 on `w10000`. That difference is what turns the median 10-pose local window from about 0.158 ms on `w20000` into about 0.958 ms on `w10000`.

The 20-pose case is the clearest example that clique counts alone are not enough. There, `w20000` actually has the larger median support count (35 cliques versus 24 on `w10000`), yet it is still faster

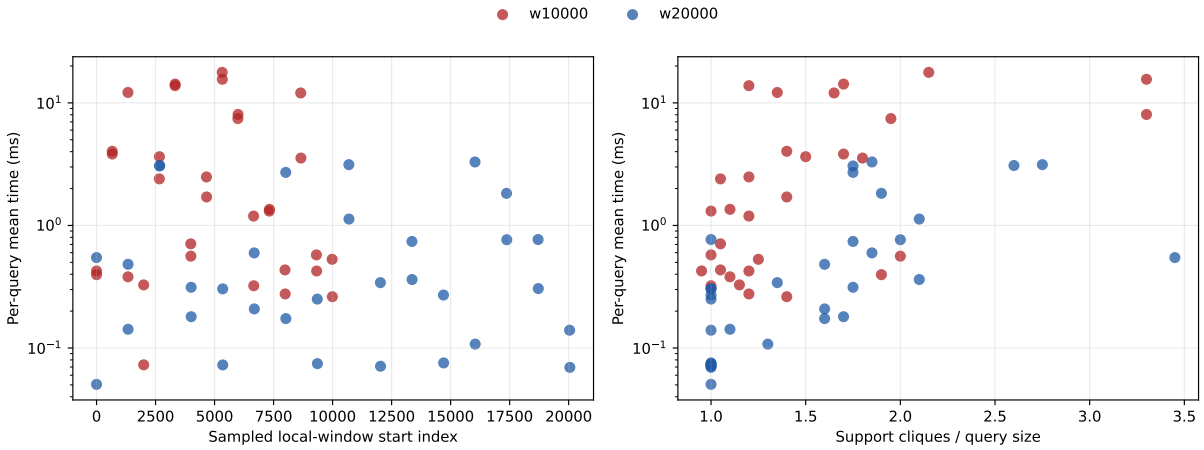

Figure 14: The revised trajectory-wide local-window samples explain why `w20000` can still be faster than `w10000`. The left panel plots per-query mean time against the sampled local-window start index, showing that the expensive `w10000` windows are spread throughout the trajectory rather than confined to the prefix used in the earlier benchmark. The right panel plots the same per-query times against support ratio. `w20000` more often stays near support ratio 1 for 10-pose windows, and for larger windows it retains much smaller separator widths even when the support count itself grows.

(0.572 ms versus 1.85 ms). The reason is again width: the median maximum separator dimension is about 18 on `w20000` but about 115.5 on `w10000`. In other words, the larger grid is not faster because it is larger. It is faster on these sampled local queries because its localized supports stay narrower in the elimination tree.

8.6 Incremental iSAM2 Querying on `w20000`

The incremental viewpoint is of independent interest because iSAM2 maintains, edits, and relinearizes a Bayes tree over time rather than working with one fixed tree (Kaess et al., 2011, 2012). For a query-local covariance method, this raises a natural question: as the tree evolves, how does the cost of a fixed pairwise covariance query evolve with it? To answer that question, we processed `w20000` sequentially, timed each iSAM2 update once, and then timed the pairwise joint covariance query $\mathrm{Cov}(x_0, x_t)$ with one warmup and five measured repetitions. Figure 15 shows the resulting traces over all 20,061 poses. The update times fluctuate much more than the query times. Over the full run, the median update time is about 0.380 ms, but it reaches a maximum of about 81.3 ms. The rolling median is only about 0.011 ms around step 100, rises to about 4.54 ms near step 5,000, and falls back below 1 ms by about step 10,000.

The covariance query is steadier. After the first short-path phase, the $\{x_0, x_t\}$ query settles into a few-millisecond band: the full-run median is about 3.48 ms, with rolling medians of about 3.94 ms near step 1,000, 3.83 ms near step 5,000, and about 2.78 ms near the end of the run. Structurally, this is because the pairwise Bayes-tree mechanism keeps the compressed problem fixed at two cliques and a 6-dimensional reduced state, even though the uncompressed support can grow long along the tree and reaches over 1,400 cliques at its maximum. The update cost therefore reflects incremental relinearization and partial re-elimination, whereas the query cost reflects traversal of an increasingly long but aggressively compressed pairwise support. This stable query behavior is consistent with the role of shortcuts: although the path from the current pose to the root changes over time, the pairwise query continually compresses that growing path to a very small reduced

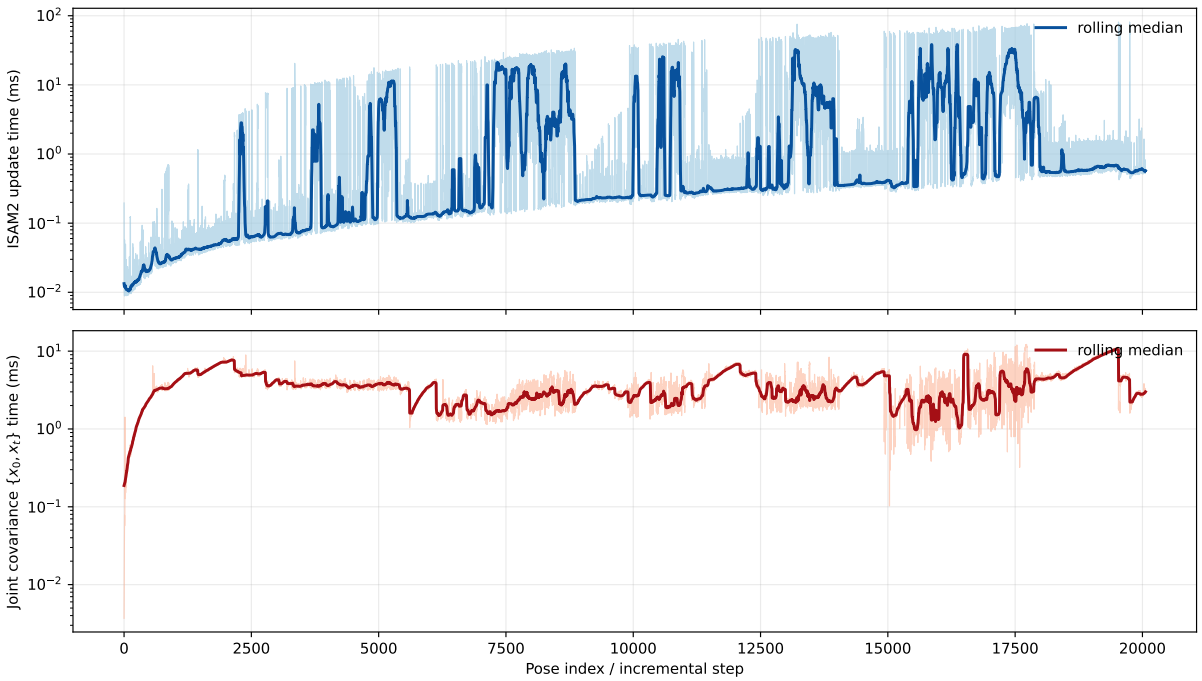

Figure 15: Sequential iSAM2 timing on `w20000` with the pairwise joint covariance query $\{x_0, x_t\}$ after every update. The light traces show raw per-step timings, while the dark curves show a rolling median over 100 steps. The update cost spikes when relinearization and re-elimination affect larger portions of the tree, whereas the covariance query remains in a much tighter few-millisecond band once the path to the current pose becomes long.

system.

9 Discussion and Future Work

We contribute both a written account of the existing Bayes-tree covariance mechanism and one new structural generalization built on top of it. The recursive one-variable and two-variable procedures have been present for years in the open-source GTSAM library (Dellaert, 2012; Dellaert and Kaess, 2017), but had remained largely implicit. Writing them down makes the method easier to analyze, compare, and extend. The central new step is exact structural localization to the Steiner support $\mathcal{T}_\mathcal{Q}$ for arbitrary query sets $\mathcal{Q}$, while solve-based extraction remains a secondary numerical refinement on the reduced covariance matrix $\Sigma_\mathcal{Q}$ after that structural localization.

One direction is to combine Bayes-tree shortcut reuse with richer selected-inverse machinery on the reduced system. The current formulation already isolates the exact query support $\mathcal{T}_\mathcal{Q}$, but repeated overlapping workloads should admit stronger reuse of both structural compression and numerical block recovery than either the historical formulation or the present query-by-query formulation.

A second direction is to develop controlled approximations on top of the exact theory. The related work on approximate sparsity and sparse-precision covariance approximation (Frese, 2005; Sidén et al., 2018) suggests that large long-range queries may admit error-controlled truncations or ordering-aware support design rules. An exact Steiner-subtree theory, now paired with a written account of the existing recursive mechanism, provides a baseline against which those approximations could be measured.

10 Conclusion

Bayes-tree covariance recovery can be understood in three layers. The first is the recursive Bayes-tree mechanism itself, which has long existed in GTSAM for one-variable and two-variable queries but had not previously been described. The second is the new structural contribution introduced here: the support of an arbitrary query set $\mathcal{Q}$ is the Steiner subtree $\mathcal{T}_\mathcal{Q}$ spanning the queried cliques, together with the generalized shortcut compression of its internal non-branching paths. The third is the secondary numerical contribution: covariance blocks should be recovered by triangular solves and block selected inversion on the reduced square-root factor $R_\mathcal{Q}$, not by dense inversion of the reduced information matrix $\Lambda_\mathcal{Q}$.

The experiments give a practical summary. Steiner localization is the main source of speedup on realistic Manhattan pose graphs, often reducing local joint-covariance queries by two to three orders of magnitude, while solve-based extraction contributes a smaller but meaningful second gain on the reduced joint systems produced by that localization. Taken together, the existing local Bayes-tree recursion, the new Steiner generalization, and the solve-based extraction step yield a query-local account of covariance recovery: local in the combinatorial sense, because only the Steiner subtree $\mathcal{T}_\mathcal{Q}$ is touched, and local in the numerical sense, because only the requested covariance blocks are computed.

References

Jean R. S. Blair and Barry Peyton. An introduction to chordal graphs and clique trees. In Alan George, John R. Gilbert, and Joseph W. H. Liu, editors, *Graph Theory and Sparse Matrix Computation*, volume 56 of *The IMA Volumes in Mathematics and its Applications*, pages 1–29. Springer, New York, 1993.

Frank Dellaert. Factor graphs and GTSAM: A hands-on introduction. *Georgia Institute of Technology, Tech. Rep*, 2(4), 2012.

Frank Dellaert and Michael Kaess. Factor graphs for robot perception. *Foundations and Trends in Robotics*, 6(1–2):1–139, August 2017. doi: 10.1561/2300000043.

A. M. Erisman and W. F. Tinney. On computing certain elements of the inverse of a sparse matrix. *Communications of the ACM*, 18(3):177–179, March 1975.

Udo Frese. A proof for the approximate sparsity of SLAM information matrices. In *Proceedings of the IEEE International Conference on Robotics and Automation, ICRA*, pages 329–335, 2005. doi: 10.1109/ROBOT.2005.1570140.

Viorela Ila, Josep M. Porta, and Juan Andrade-Cetto. Amortized constant time state estimation in pose SLAM and hierarchical SLAM using a mixed Kalman-information filter. *Robotics and Autonomous Systems*, 59(5):310–318, May 2011. doi: 10.1016/j.robot.2011.02.010.

M. Kaess and F. Dellaert. Covariance recovery from a square root information matrix for data association. *J. of Robotics and Autonomous Systems, RAS*, 57(12):1198–1210, December 2009.

M. Kaess, V. Ila, R. Roberts, and F. Dellaert. The Bayes tree: An algorithmic foundation for probabilistic robot mapping. In *Proc. Intl. Workshop on the Algorithmic Foundations of Robotics, WAFR*, pages 157–173, Singapore, December 2010.

M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert. iSAM2: Incremental smoothing and mapping with fluid relinearization and incremental variable reordering. In *Proceedings of the IEEE International Conference on Robotics and Automation, ICRA*, pages 3281–3288, May 2011. doi: 10.1109/ICRA.2011.5979641.

M. Kaess, H. Johannsson, R. Roberts, V. Ila, J.J. Leonard, and F. Dellaert. iSAM2: Incremental smoothing and mapping using the Bayes tree. *Intl. J. of Robotics Research, IJRR*, 31(2):216–235, February 2012.

Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, MA, 2009.

Lin Lin, Chao Yang, Juan C. Meza, Jianfeng Lu, Lexing Ying, and Weinan E. SelInv—an algorithm for selected inversion of a sparse symmetric matrix. *ACM Transactions on Mathematical Software*, 37(4):40:1–40:19, February 2011. doi: 10.1145/1916461.1916464.

Edwin Olson, John J. Leonard, and Seth J. Teller. Fast iterative alignment of pose graphs with poor initial estimates. In *Proceedings of the IEEE International Conference on Robotics and Automation, ICRA*, pages 2262–2269, 2006. doi: 10.1109/ROBOT.2006.1642040.

Per Sidén, Finn Lindgren, David Bolin, and Mattias Villani. Efficient covariance approximations for large sparse precision matrices. *Journal of Computational and Graphical Statistics*, 27(4): 898–909, August 2018. doi: 10.1080/10618600.2018.1473782.

Andrew Zammit-Mangion and Jonathan Rougier. A sparse linear algebra algorithm for fast computation of prediction variances with Gaussian Markov random fields. *Computational Statistics & Data Analysis*, 123:116–130, July 2018. doi: 10.1016/j.csda.2018.02.001.

Acknowledgements

This report used Codex GPT-5.4 for both drafting and revising the report. Thanks to Florian Meyer for reading early drafts.